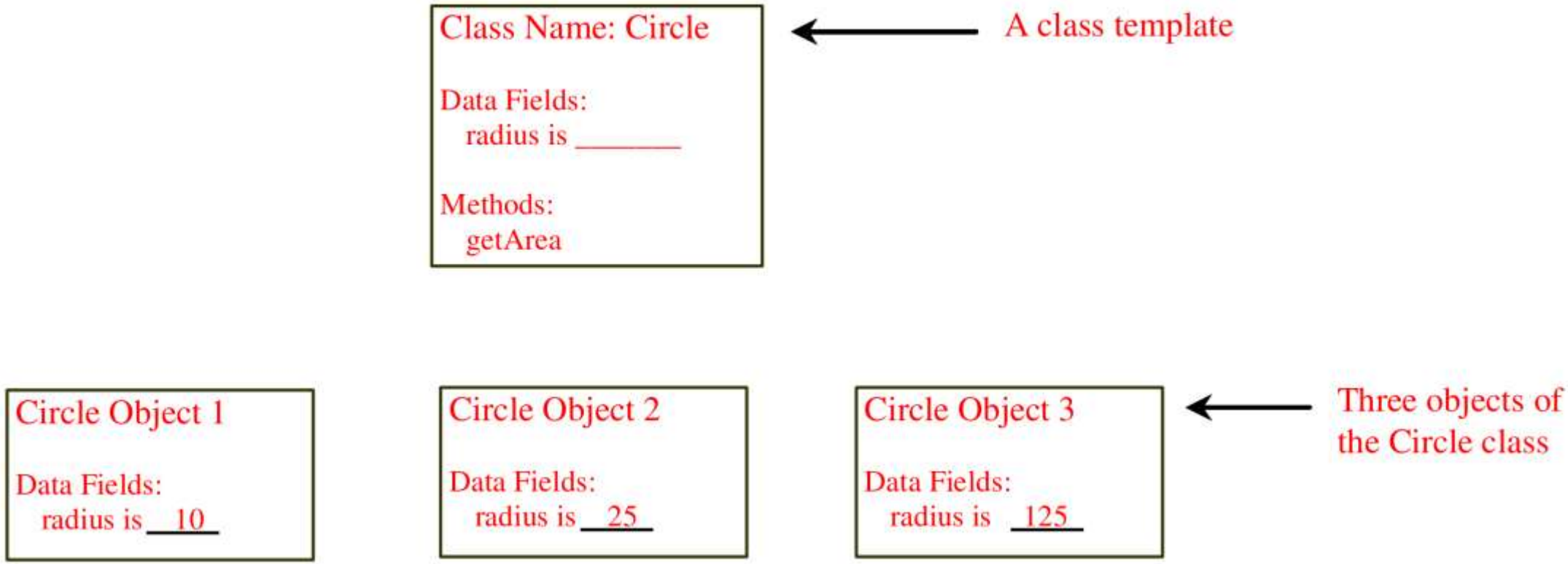
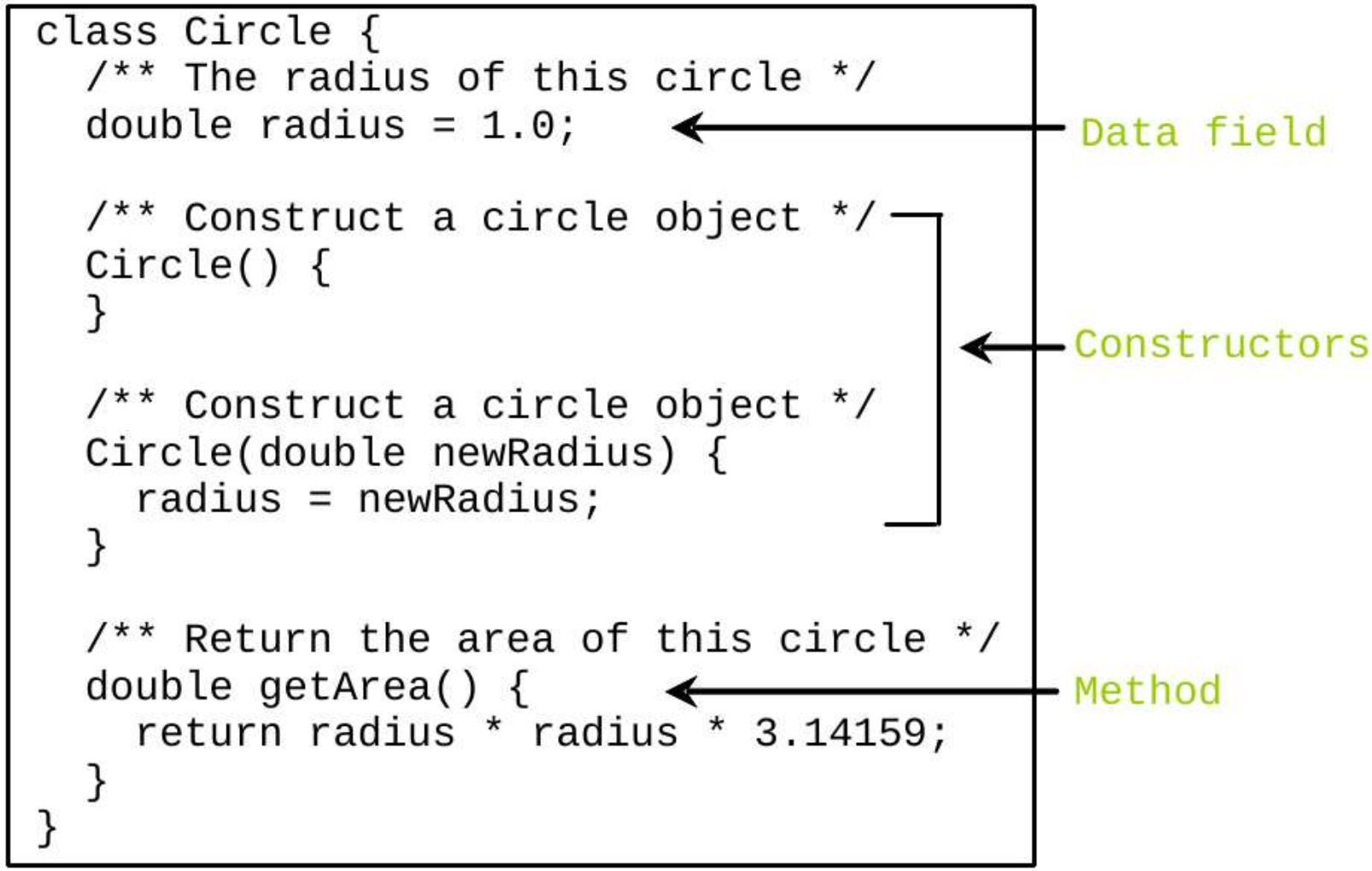
Objects and Classes

Objects



An object has both a **state** and **behavior**. The state defines the object, and the behavior defines what the object does.

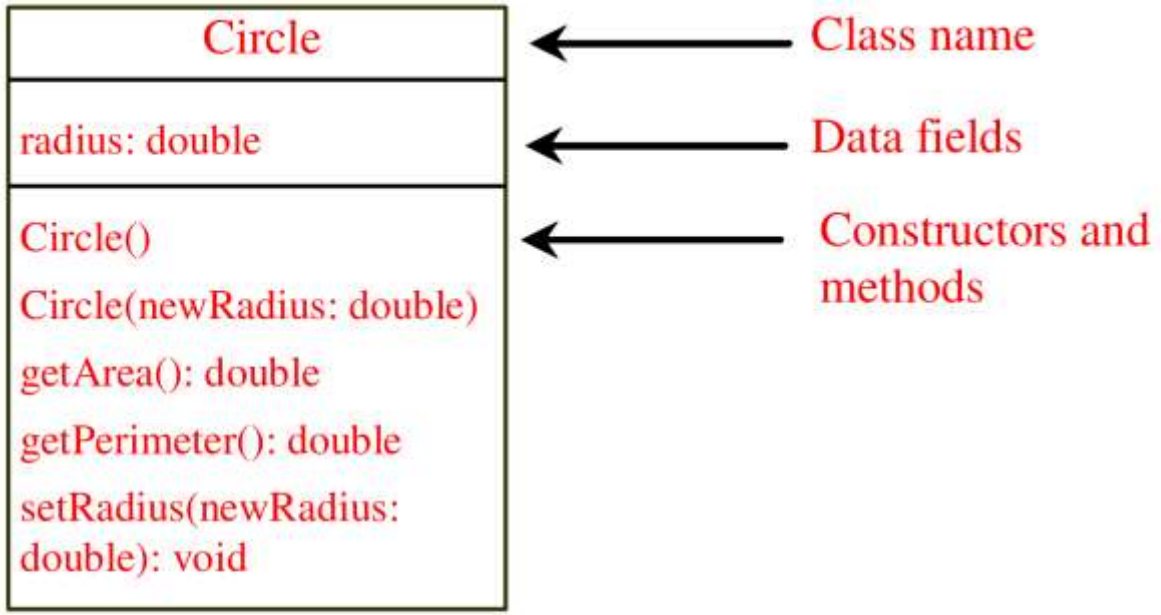
Classes


class Circle {
/** The radius of this circle */
double radius = 1.0;
/** Construct a circle object */
Circle() {
}
/** Construct a circle object */
Circle(double newRadius) {
radius = newRadius;
}
/** Return the area of this circle */
double getArea() {
return radius * radius * 3.14159;
}
}
Data field
Constructors
Method

UML Class Diagram

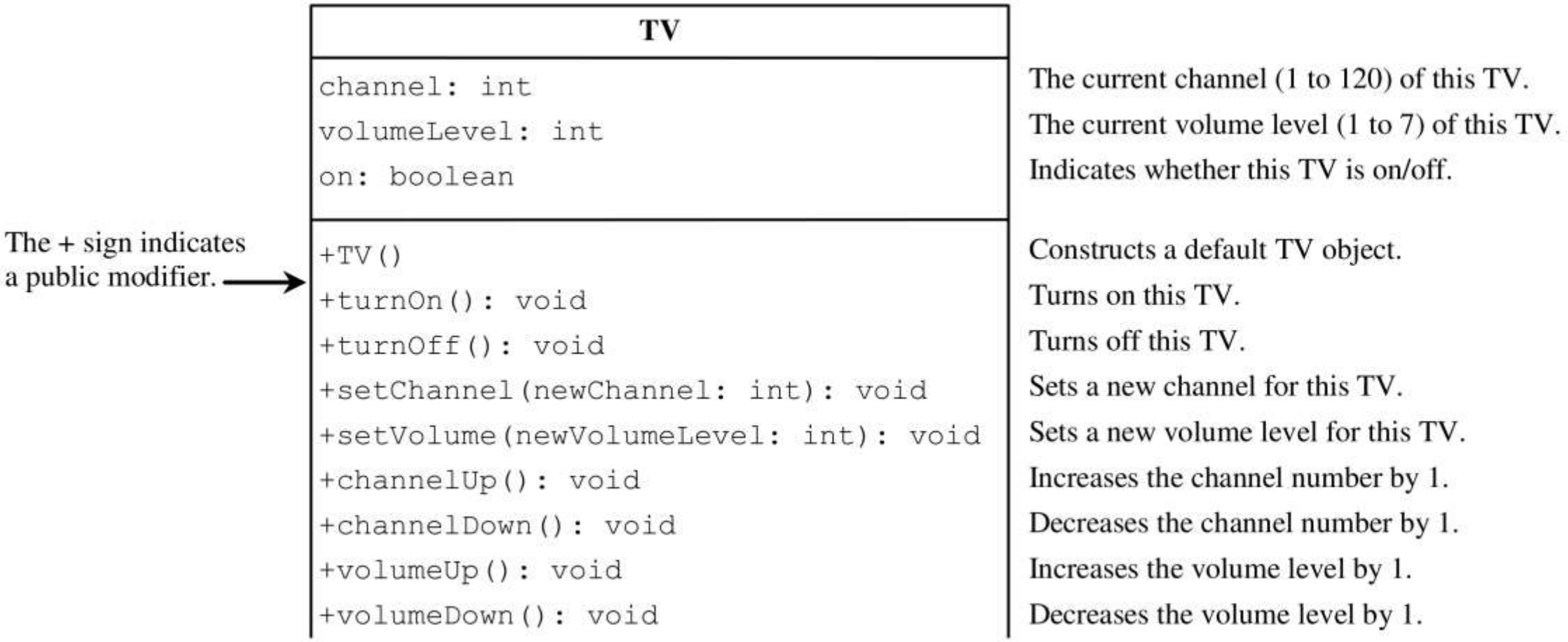
UML Class Diagram

(Unified Modeling Language)





Example: Defining Classes and Creating Objects



Example: Defining Classes and Creating Objects

```java
public class TV {
  int channel = 1; // Default channel is 1
  int volumeLevel = 1; // Default volume level is 1
  boolean on = false; // By default TV is off

  public TV() {
  }

  public void turnOn() {
    on = true;
  }

  public void turnOff() {
    on = false;
  }

  public void setChannel(int newChannel) {
    if (on && newChannel >= 1 && newChannel <= 120)
      channel = newChannel;
  }

  public void setVolume(int newVolumeLevel) {
    if (on && newVolumeLevel >= 1 && newVolumeLevel <= 7)
      volumeLevel = newVolumeLevel;
  }

  public void channelUp() {
    if (on && channel < 120)
      channel++;
  }

  public void channelDown() {
    if (on && channel > 1)
      channel--;
  }

  public void volumeUp() {
    if (on && volumeLevel < 7)
      volumeLevel++;
  }

  public void volumeDown() {
    if (on && volumeLevel > 1)
      volumeLevel--;
  }
}
```

```java
public class TestTV {
  public static void main(String[] args) {
    TV tv1 = new TV(); // Create a TV
    tv1.turnOn(); // Turn on tv1
    tv1.setChannel(30);
    tv1.setVolume(3);

    TV tv2 = new TV();
    tv2.turnOn();
    tv2.channelUp();
    tv2.channelUp();
    tv2.volumeUp(); // Increase tv2 volume up 1 level

    System.out.println("tv1's channel is " + tv1.channel
      + " and volume level is " + tv1.volumeLevel);
    System.out.println("tv2's channel is " + tv2.channel
      + " and volume level is " + tv2.volumeLevel);
  }
}
```

```
<terminated> TestTV [Java Application] C:\Program Files\Java\jr
tv1's channel is 30 and volume level is 3
tv2's channel is 3 and volume level is 2
```

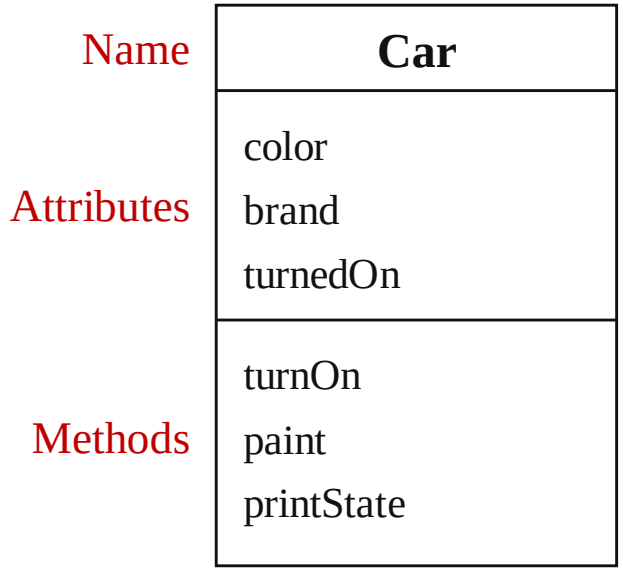
Class – definition

```
public class Car {
  String color;
  String brand;
  boolean turnedOn;

  void turnOn() {
    turnedOn = true;
  }

  void paint (String newCol) {
    color = newCol;
  }

  void printState () {
    System.out.println("Car " + brand + " " + color);
    System.out.println("the engine is"
      +(turnedOn?"on":"off"));
  }
}
```



Constructors

```
Circle() {
}

Circle(double newRadius) {
  radius = newRadius;
}
```

Constructors are a special kind of methods that are invoked to construct objects.

A constructor with no parameters is referred to as a no-arg constructor.

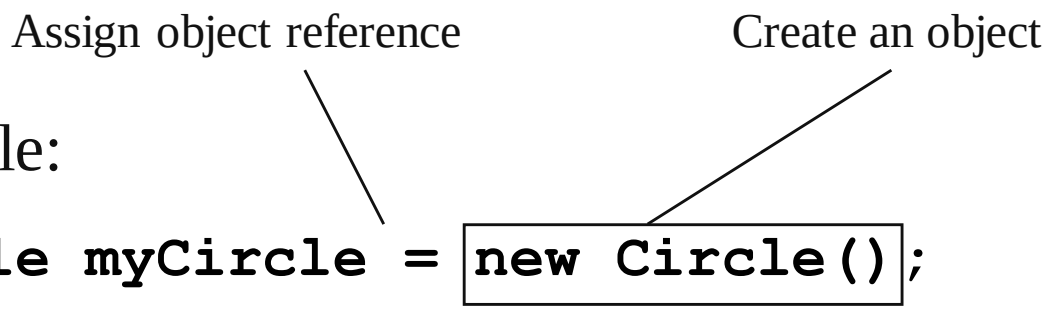
- Constructors must have the same name as the class itself.
- Constructors do not have a return type - not even **`void`**.
- Attributes are always initialized (with default values if not specified)
- Constructors are invoked using the **`new`** operator when an object is created.

The keyword `new`

- Creates a new instance of the specific class
- Allocates the required memory
- Calls the constructor of the object
- Returns a reference to the new object
- Constructor may have parameters

Declaring/Creating Objects in a Single Step

```
ClassName objectRefVar = new ClassName();
```



Example:

```
Circle myCircle = new Circle();
```

Accessing Objects

- Referencing the object's data:

```
objectRefVar.data
e.g., myCircle.radius
```

- Invoking the object's method:

```
objectRefVar.methodName(arguments)
e.g., myCircle.getArea()
```

Access to Attributes of the Same Object

- Methods accessing attributes of the same object do not need to use the object reference

```
class Car {
  String color;
  …
  void paint(){
    color = "green";
    // color refers to current obj
  }
}
```

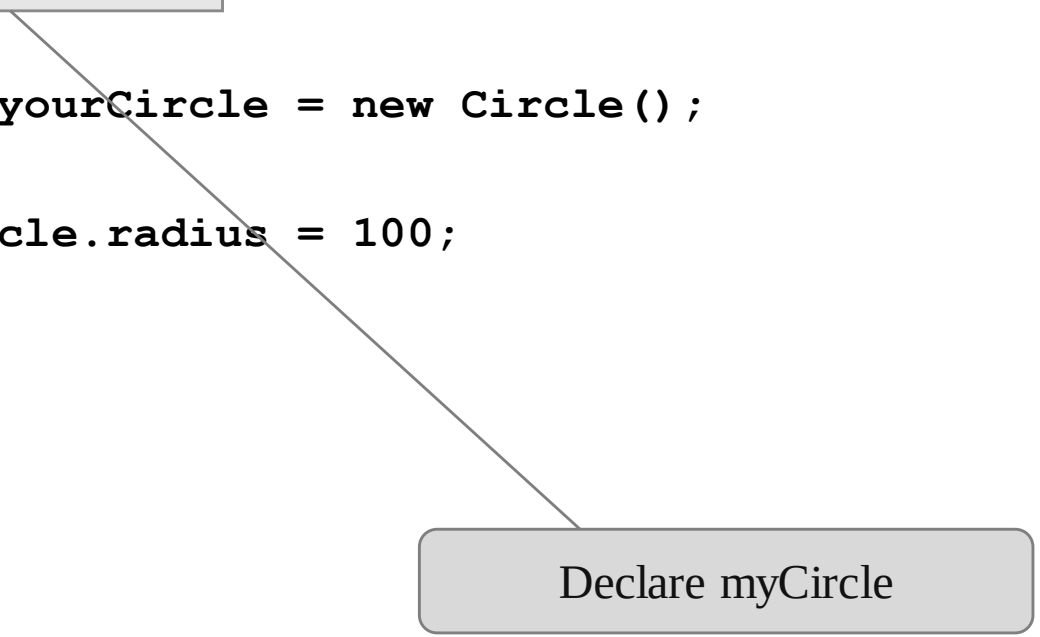
Trace Code

myCircle | no value

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;
```

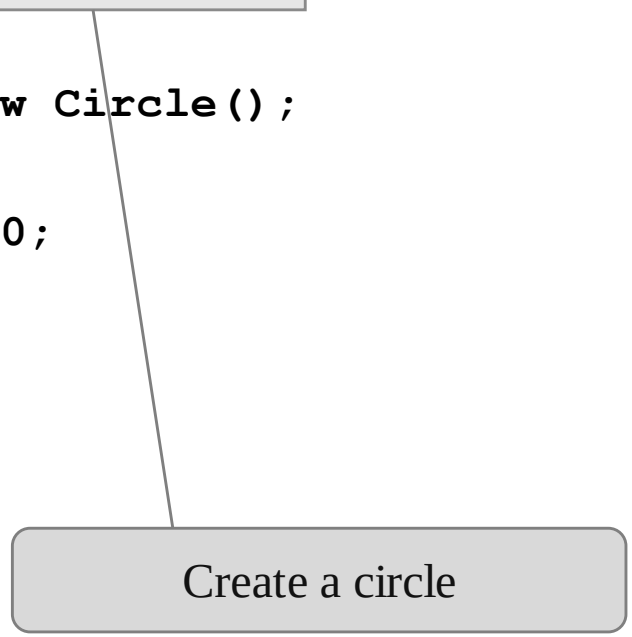


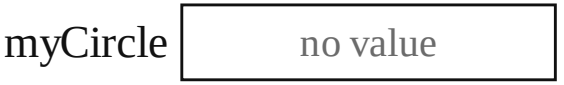
Trace Code

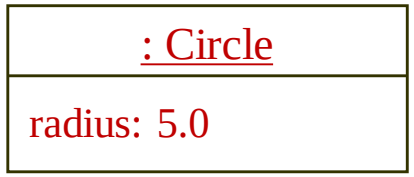
```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;
```







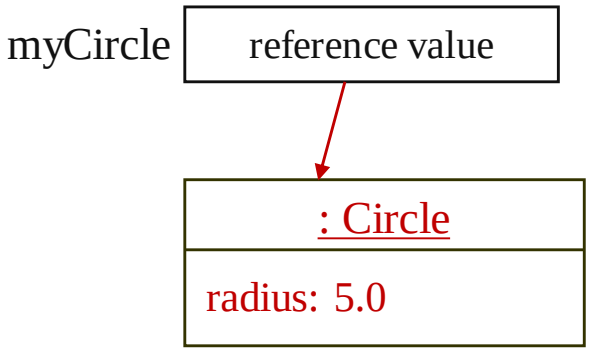
Trace Code

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;
```

Assign object reference to myCircle

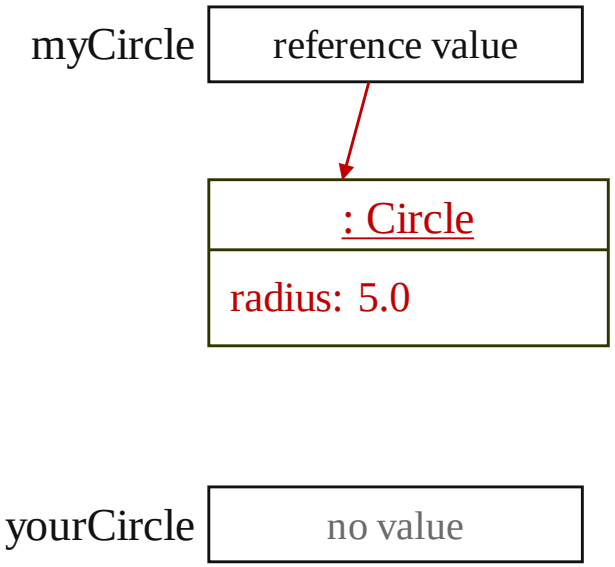


Trace Code

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;
```



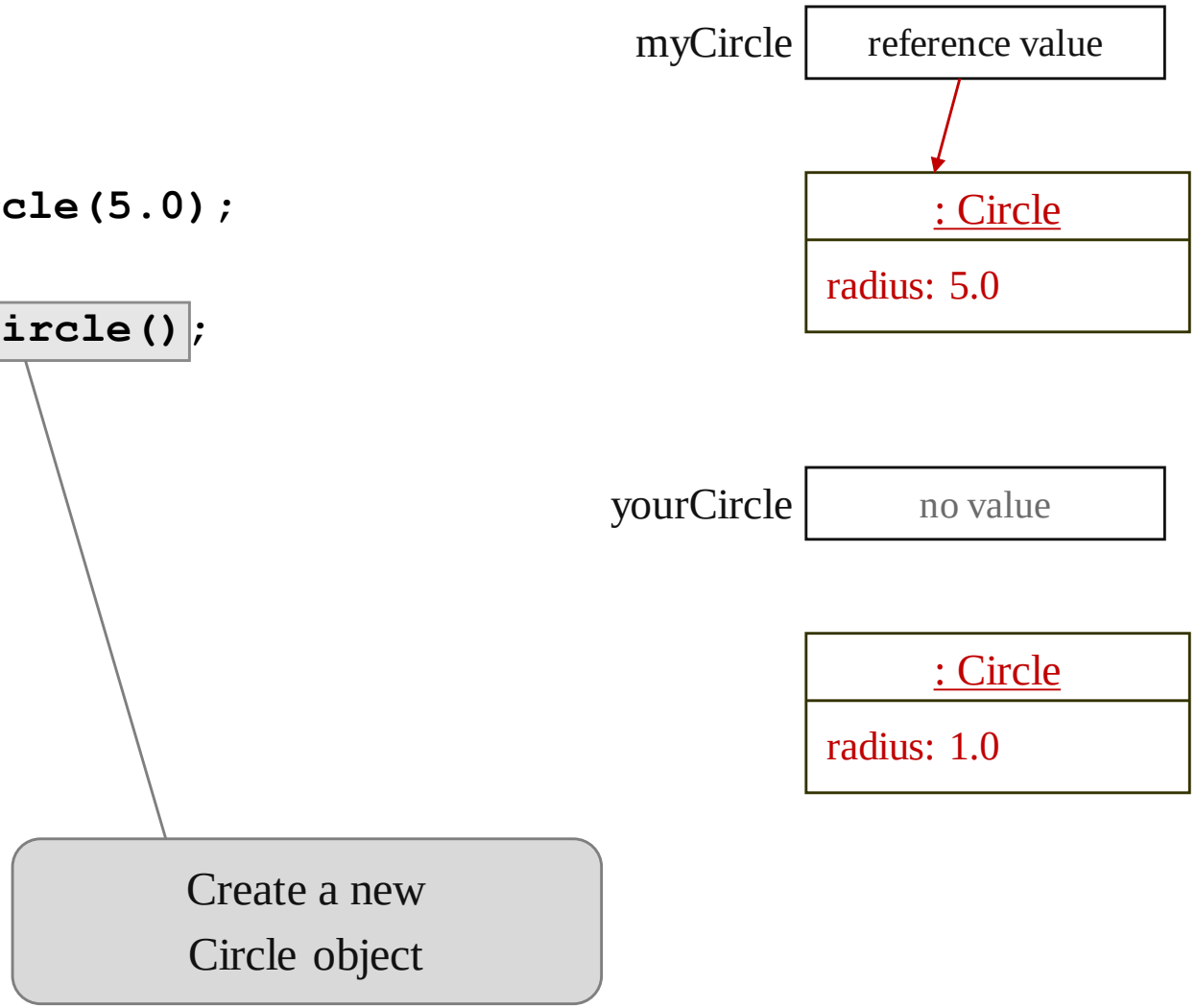
Declare yourCircle

Trace Code

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;
```

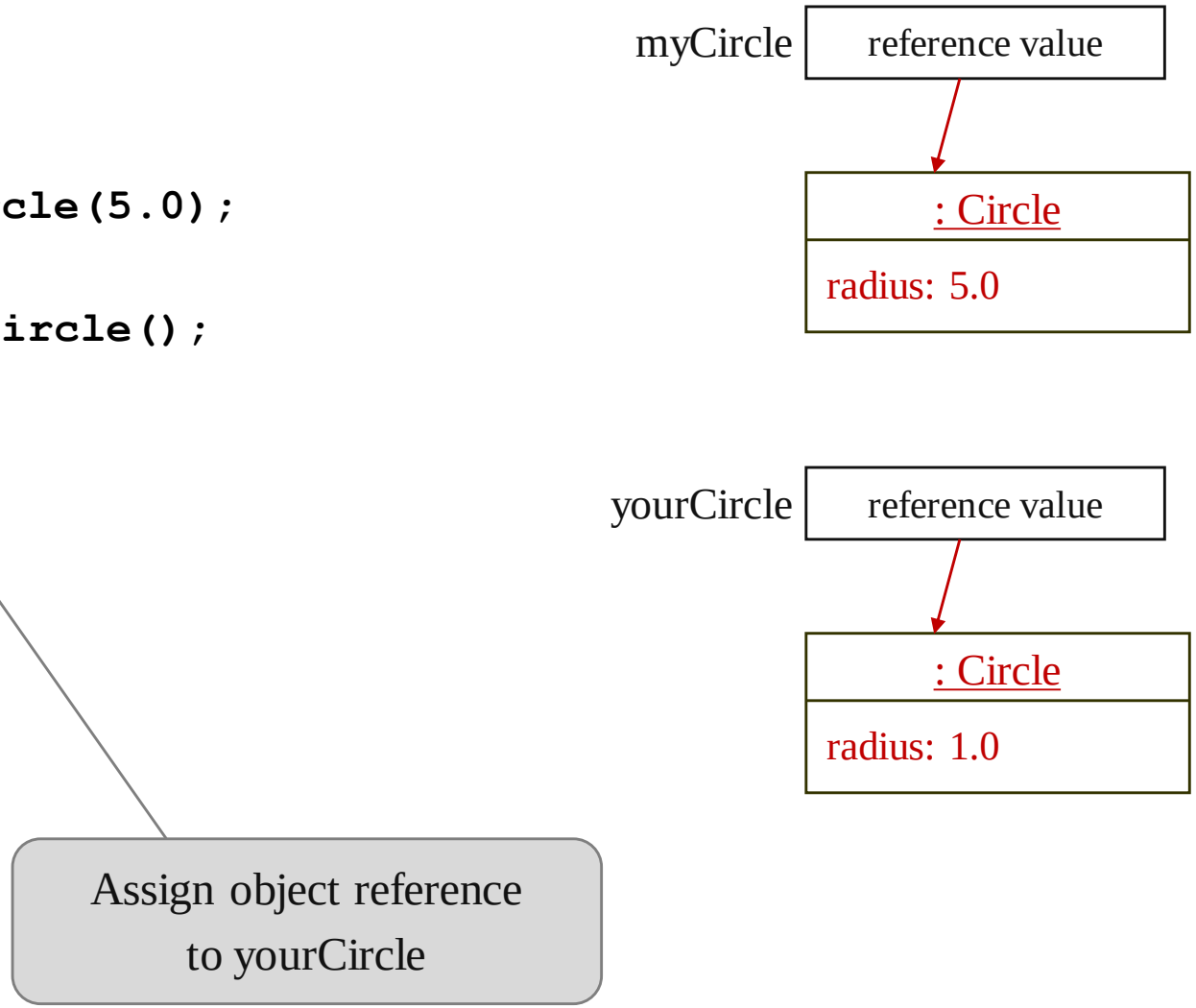


Trace Code

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;
```

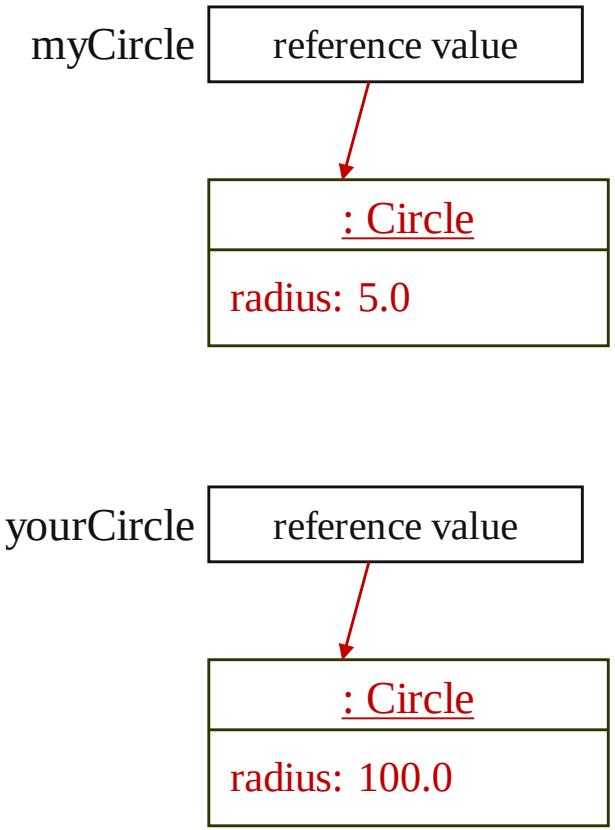


Trace Code

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;
```



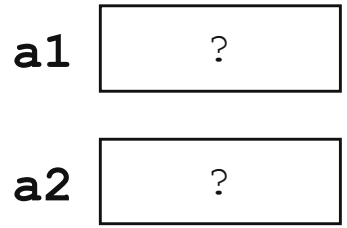
Change radius in yourCircle

Objects and References

- An object can be accessed through a reference
  - Any object can be pointed to by one or more references
  - Aliasing

Objects and References

```
Car a1, a2;
a1 = new Car();
a1.paint("yellow");
a2 = a1;
a1 = null;
a2 = null;
```

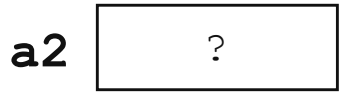


Two uninitialized references are created, they can't be used in any way.
A reference is not an object

Objects and References

```
Car a1, a2;
a1 = new Car();
a1.paint("yellow");
a2 = a1;
a1 = null;
a2 = null;
```



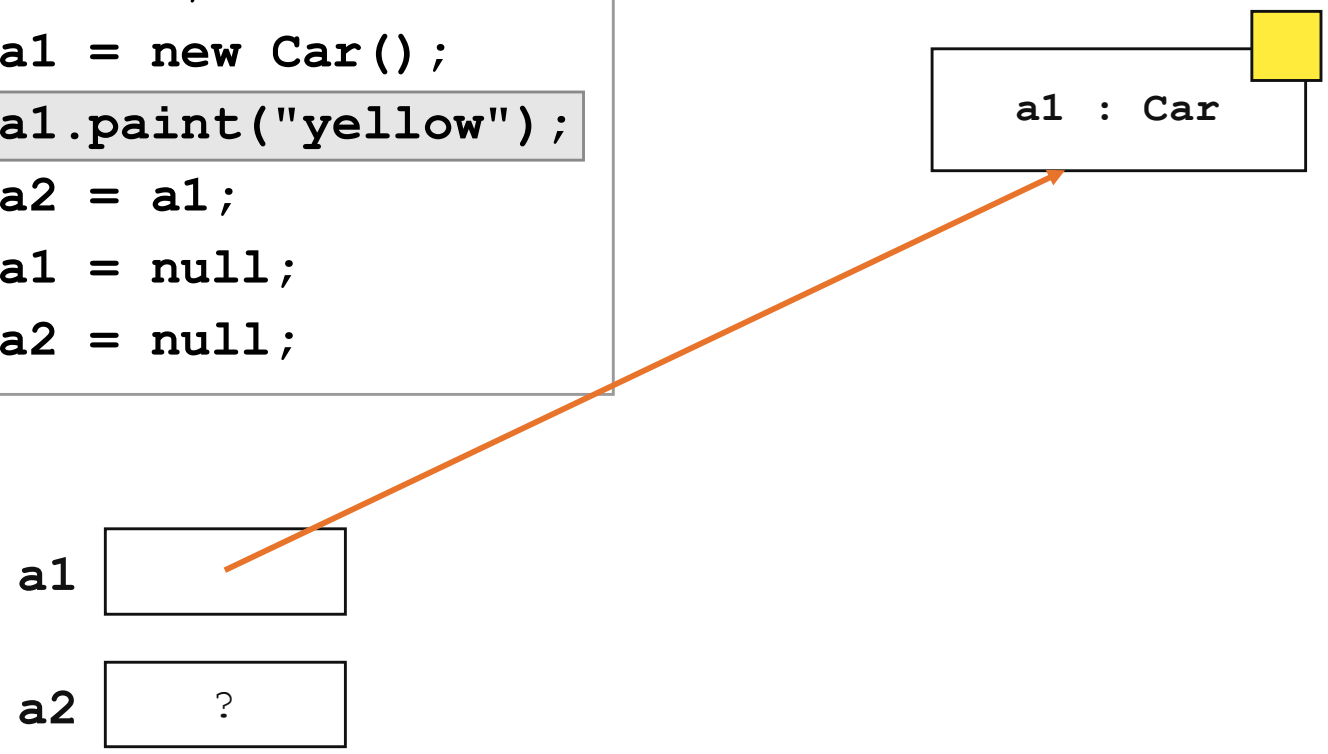




An object is created
and the “pointer” stored into
the reference a1

Objects and References

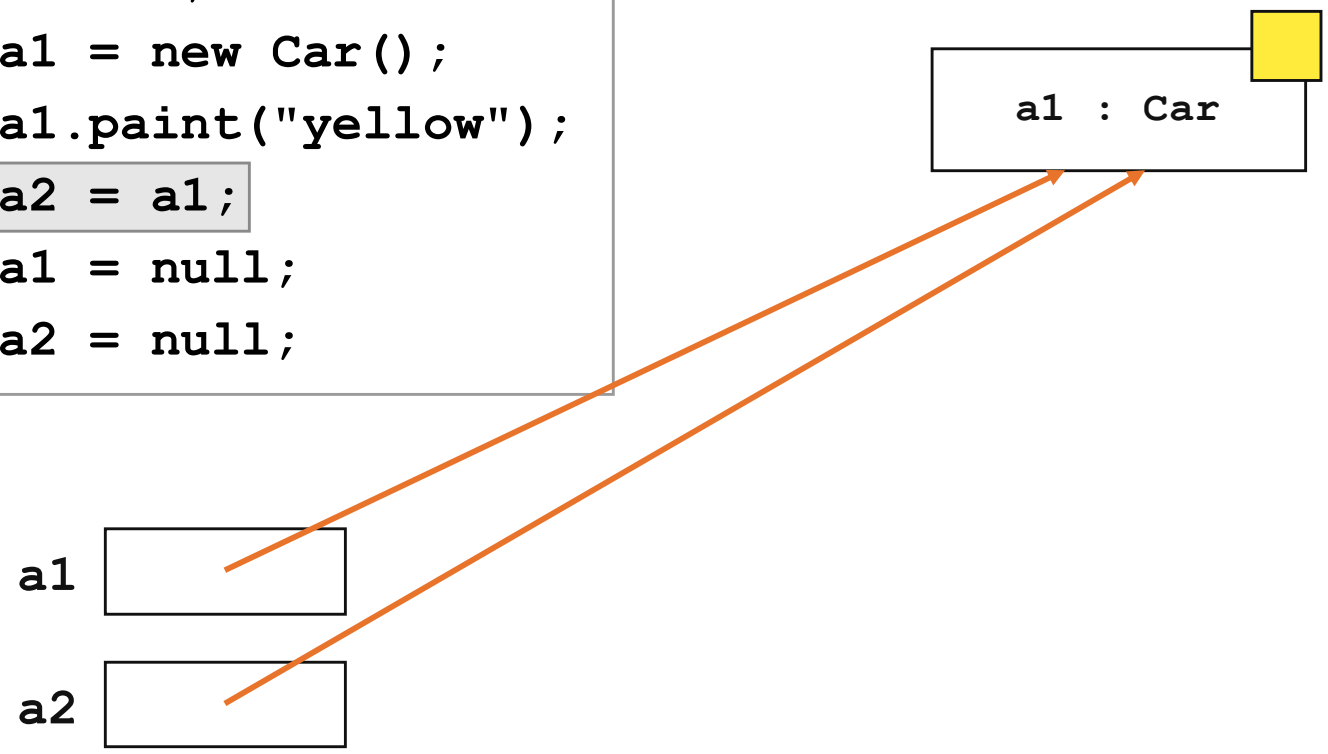
```
Car a1, a2;
a1 = new Car();
a1.paint("yellow");
a2 = a1;
a1 = null;
a2 = null;
```



Method paint() is invoked on the object through the reference a1

Objects and References

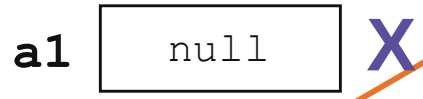
```
Car a1, a2;
a1 = new Car();
a1.paint("yellow");
a2 = a1;
a1 = null;
a2 = null;
```



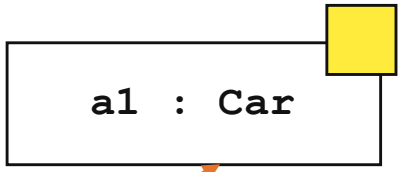
Two references point
to the same object.
This is **aliasing**

Objects and References

```
Car a1, a2;
a1 = new Car();
a1.paint("yellow");
a2 = a1;
a1 = null;
a2 = null;
```



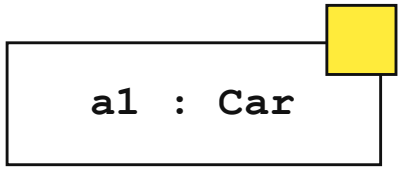




Only one reference points to the object

Objects and References

```
Car a1, a2;
a1 = new Car();
a1.paint("yellow");
a2 = a1;
a1 = null;
a2 = null;
```





No reference pointing to the object, which is unreachable and may be disposed of by the **garbage collector**

Operations on References

- Only the comparison operators `==` and `!=` are defined
  - The equality condition is evaluated on the values of the references and NOT on the objects themselves!
  - The relational operators tells whether the references points to the same object in memory
- There is NO pointer arithmetic

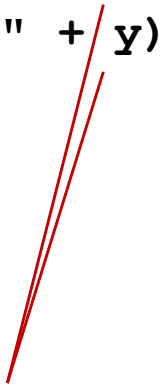
Default Value for a Data Field

The default value of a data field is **`null`** for a reference type, 0 for a numeric type, **`false`** for a **`boolean`** type, and **`'\u0000'`** for a **`char`** type.

```
public class Test {
  public static void main(String[] args) {
    Student student = new Student();
    System.out.println("name? " + student.name);
    System.out.println("age? " + student.age);
    System.out.println("isScienceMajor? " + student.isScienceMajor);
    System.out.println("gender? " + student.gender);
  }
}
```

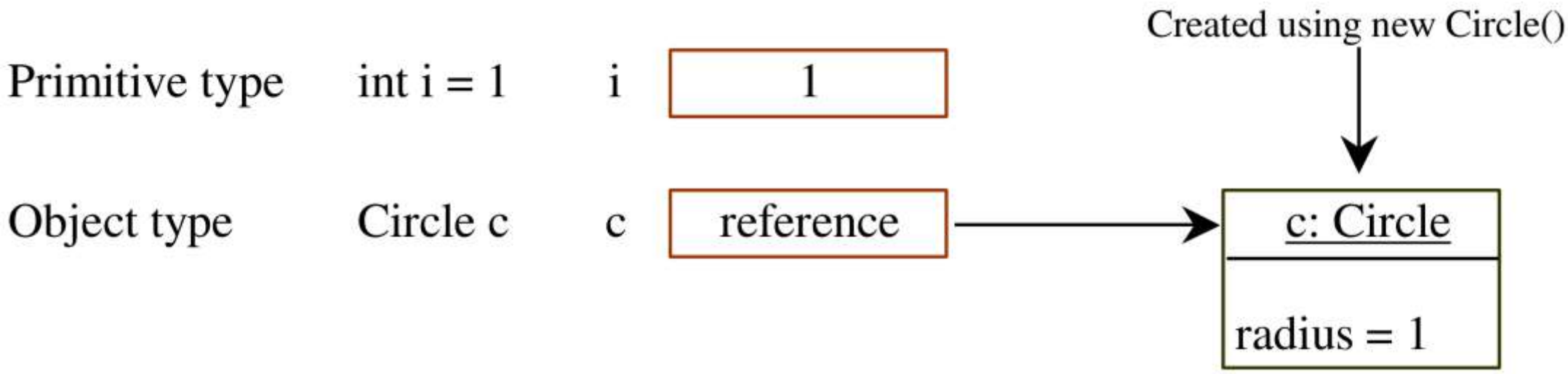
However, Java assigns no default value to a local variable inside a method.

```
public class Test {
  public static void main(String[] args) {
    int x; // x has no default value
    String y; // y has no default value
    System.out.println("x is " + x);
    System.out.println("y is " + y);
  }
}
```



Compile error: variable not initialized

Differences between Variables of Primitive Data Types and Object Types

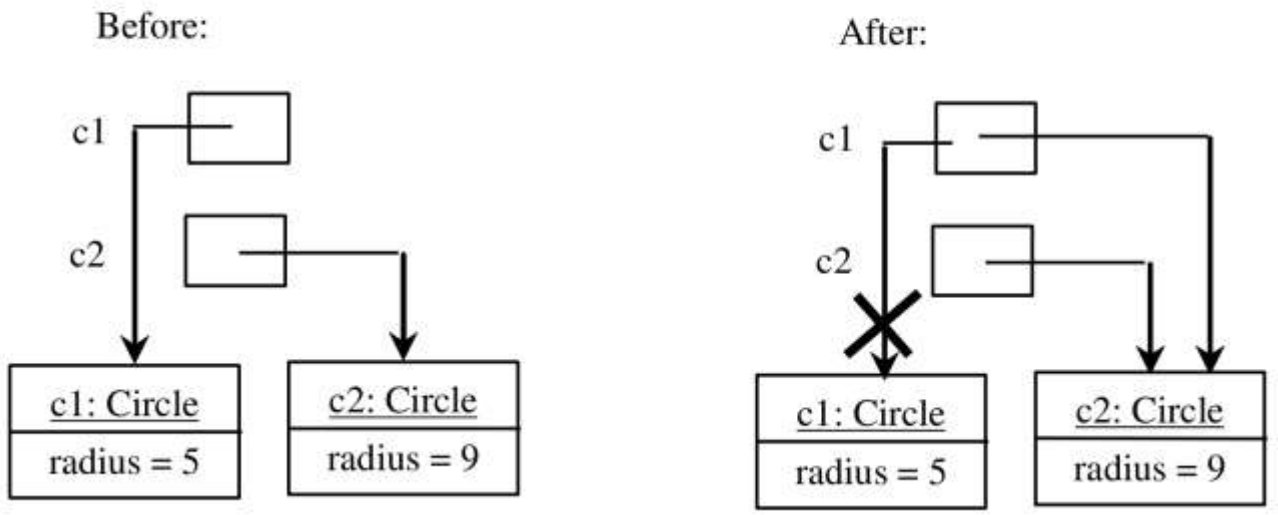


Copying Variables of Primitive Data Types and Object Types

Primitive type assignment i = j

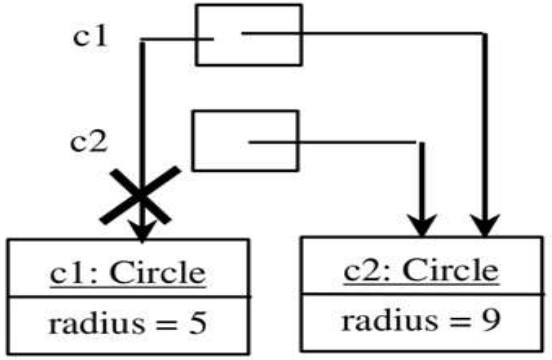


Object type assignment c1 = c2



Garbage Collection

As shown in the figure, after the assignment statement **`c1 = c2`**, **`c1`** points to the same object referenced by **`c2`**. The object previously referenced by **`c1`** is no longer referenced. This object is known as garbage. Garbage is automatically collected by JVM. Memory release, in Java, is not a programmer's concern



TIP: If you know that an object is no longer needed, you can explicitly assign **`null`** to a reference variable for the object. The JVM will automatically collect the space if the object is not referenced by any variable.

Static Variables, Constants, and Methods

Static variables are shared by **all** the instances of the class.

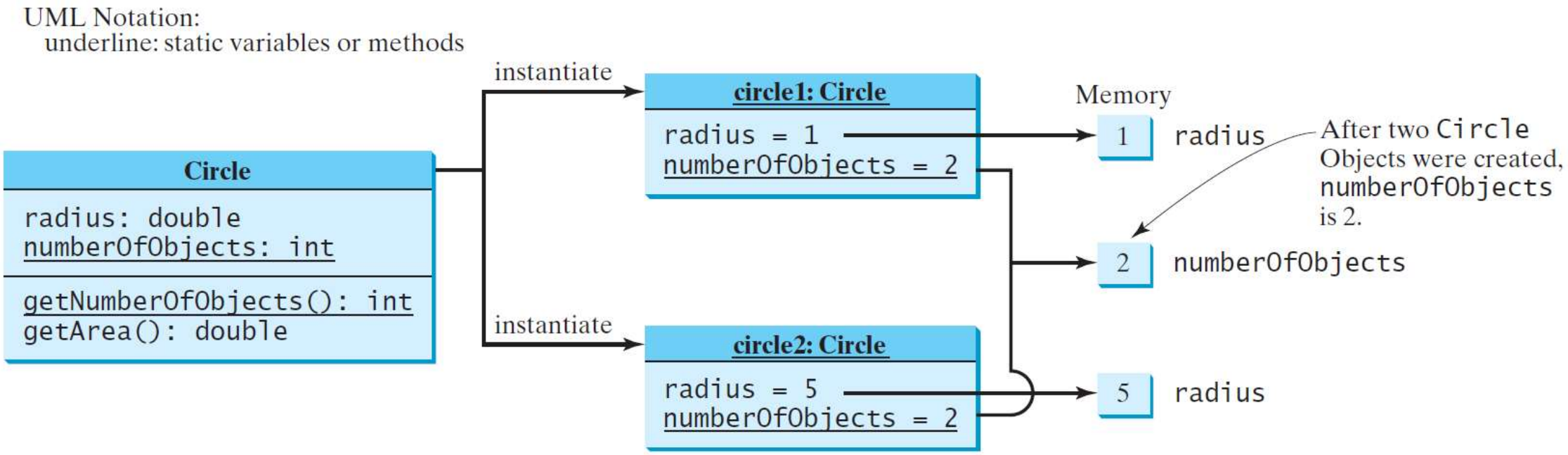
Static methods are not tied to a specific object.

Static constants are final variables shared by all the instances of the class.

To declare static variables, constants, and methods, use the **`static`** modifier.

```
static int numberOfObjects = 0;
```

Static Variables, Constants, and Methods, cont.



Example of Static Variable and Method

```
public class CircleWithStaticMembers {

  double radius;

  static int numberOfObjects = 0;

  CircleWithStaticMembers() {
    radius = 1.0;
    numberOfObjects++;
  }

  CircleWithStaticMembers(double newRadius) {
    radius = newRadius;
    numberOfObjects++;
  }

  /** Return numberOfObjects */
  static int getNumberOfObjects() {
    return numberOfObjects;
  }

  /** Return the area of this circle */
  double getArea() {
    return radius * radius * Math.PI;
  }
}
```

Example of Static Variable and Method cont.

```
public static void main(String[] args) {
  System.out.println("Before creating objects");
  System.out.println("The number of Circle objects is " + CircleWithStaticMembers.numberOfObjects);

  // Create c1
  CircleWithStaticMembers c1 = new CircleWithStaticMembers();

  // Display c1 BEFORE c2 is created
  System.out.println("\nAfter creating c1");
  System.out.println("c1: radius (" + c1.radius + ") and number of Circle objects (" + c1.numberOfObjects + ")");

  // Create c2
  CircleWithStaticMembers c2 = new CircleWithStaticMembers(5);

  // Modify c1
  c1.radius = 9;

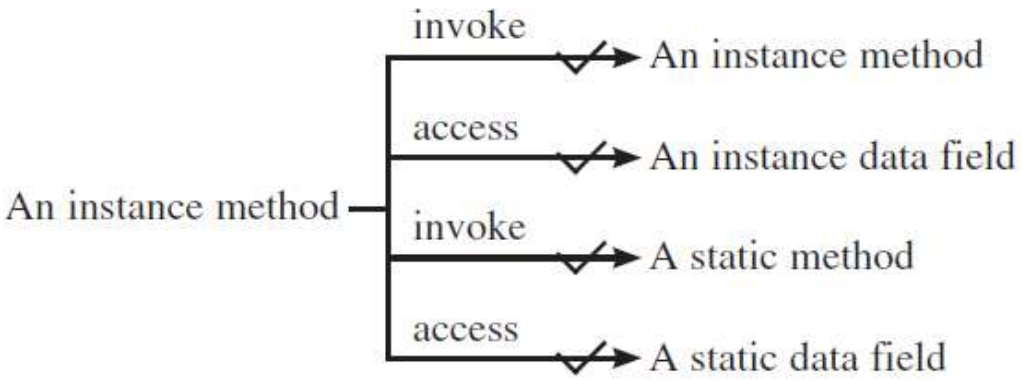
  // Display c1 and c2 AFTER c2 was created
  System.out.println("\nAfter creating c2 and modifying c1");
  System.out.println("c1: radius (" + c1.radius + ") and number of Circle objects (" + c1.numberOfObjects + ")");
  System.out.println("c2: radius (" + c2.radius + ") and number of Circle objects (" + c2.numberOfObjects + ")");
}
```

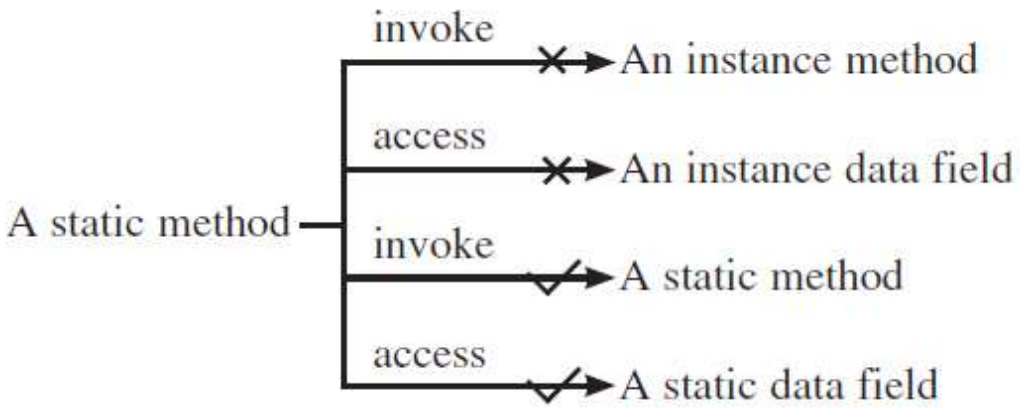
```
Before creating objects
The number of Circle objects is 0

After creating c1
c1: radius (1.0) and number of Circle objects (1)

After creating c2 and modifying c1
c1: radius (9.0) and number of Circle objects (2)
c2: radius (5.0) and number of Circle objects (2)
```

An instance method vs A static method





An instance method vs A static method

For example, the following code is wrong.

```
public class A {
  int i = 5;
  static int k = 2;

  public static void main(String[] args) {
    int j = i; // Wrong because i is an instance variable
    m1(); // Wrong because m1() is an instance method
  }

  public void m1() {
    // Correct since instance and static variables and methods can be used in an instance method
    i = i + k + m2(i, k);
  }

  public static int m2(int i, int j) {
    return (int)(Math.pow(i, j));
  }
}
```

An instance method vs A static method

If you replace the preceding code with the following new code, the program would be fine,

```
public class A {
  int i = 5;
  static int k = 2;

  public static void main(String[] args) {
    A a = new A();
    int j = a.i; // OK, a.i accesses the object's instance variable
    a.m1(); // OK. a.m1() invokes the object's instance method
  }

  public void m1() {
    i = i + k + m2(i, k);
  }

  public static int m2(int i, int j) {
    return (int)(Math.pow(i, j));
  }
}
```

Static Attributes

- Sometimes called **class attributes** as opposed to instance attributes
- Static attributes exists **before** any object is created
- A change performed by any object is **visible to all instances** at once
- Used to keep **a shared property**

```
class Car {
  static int countBuiltCars = 0;
  public Car(){
    countBuiltCars++;
  }
}
```

Static Methods: why

- Used to implement functions
  - Avoid creating an object just to invoke the method (see e.g., **`main()`**)

```
public class Utility {
  public static int inverse(double n){
    return 1 / n;
  }
}
```

Final Attributes

- An attribute declared as **`final`**:
  - **cannot be changed** after object construction
  - can be initialized inline or by the constructor

```
class Student {
  final int years=3;
  final String id;
  public Student(String id){
    this.id = id;
  }
}
```

Visibility Modifiers and Accessor/Mutator Methods

By default, the class, variable or method can be accessed by any class in **the same package**.

- **`public`**

  The class, data, or method is visible to **any class** in any package.

- **`private`**

  The data or methods can be accessed only by **the declaring class**.

The **get** and **set** methods are used to read and modify private properties.

Visibility Modifiers and Accessor/Mutator Methods

```
package p1;

public class C1 {
  public int x;
  int y;
  private int z;

  public void m1() {
  }
  void m2() {
  }
  private void m3() {
  }
}
```

```
package p1;

public class C2 {
  void aMethod() {
    C1 o = new C1();
    can access o.x;
    can access o.y;
    cannot access o.z;

    can invoke o.m1();
    can invoke o.m2();
    cannot invoke o.m3();
  }
}
```

```
package p2;

public class C3 {
  void aMethod() {
    C1 o = new C1();
    can access o.x;
    cannot access o.y;
    cannot access o.z;

    can invoke o.m1();
    cannot invoke o.m2();
    cannot invoke o.m3();
  }
}
```

The **`private`** modifier restricts access to within a class, the default modifier restricts access to within a package, and the **`public`** modifier enables unrestricted access.

Visibility Modifiers and Accessor/Mutator Methods

```
package p1;

class C1 {
  ...
}
```

```
package p1;

public class C2 {
  can access C1
}
```

```
package p2;

public class C3 {
  cannot access C1;
  can access C2;
}
```

NOTE

An object cannot access its private members, as shown in (b). It is OK, however, if the object is declared in its own class, as shown in (a).

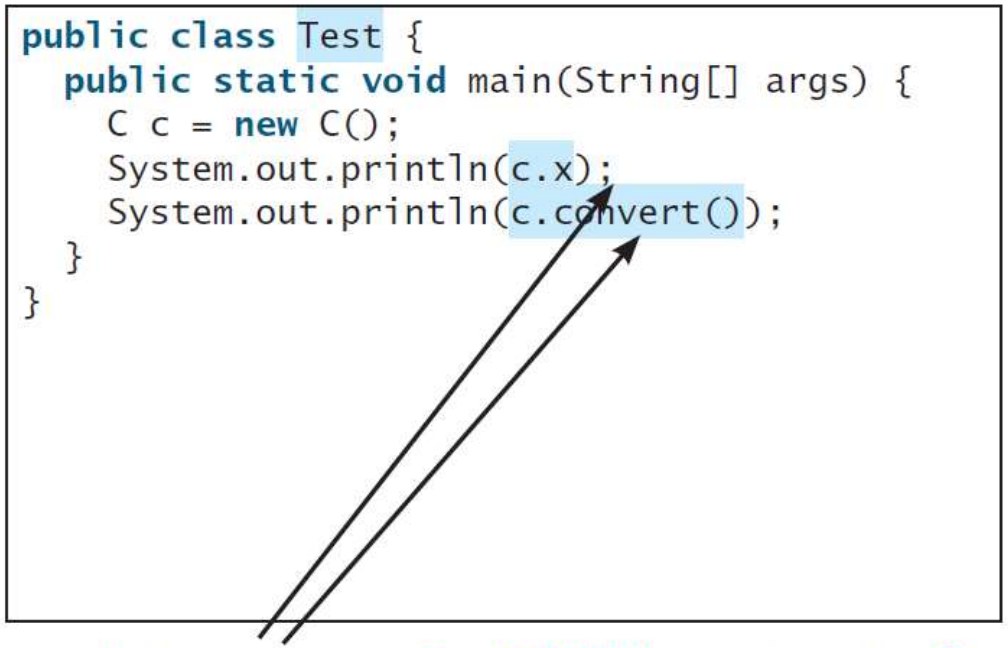
```
public class C {
  private boolean x;

  public static void main(String[] args) {
    C c = new C();
    System.out.println(c.x);
    System.out.println(c.convert());
  }

  private int convert() {
    return x ? 1 : -1;
  }
}
```

(a) This is okay because object `c` is used inside the class `C`.

```
public class Test {
  public static void main(String[] args) {
    C c = new C();
    System.out.println(c.x);
    System.out.println(c.convert());
  }
}
```



(b) This is wrong because `x` and `convert` are private in class `C`.

Package

- For the purpose of code organization and structuring Java provides the package feature
- A package is a logic set of class definitions
- These classes consist of several files, all stored in the same folder
- Each package defines a new scope. It is therefore possible to use same class names in different package without name-conflicts
- Declaration:
  - Package statement at the beginning of each class file

```
package packageName;
```

- Usage:
  - Import statement at the beginning of class file (where needed)

```
import packageName.className;
```

Visibility with Multiple Packages

- **`public class A { }`**
  - Class and public members of A are visible from outside the package
- **`class B { }`** (package visibility)
  - Class and any members of B are not visible from outside the package
- **`private class A { }`**
  - Illegal: why?

The class and its members would be visible to themselves only

Why Data Fields Should Be private?

To protect data.

To make code easy to maintain.

Encapsulation is one of the four fundamental OOP concepts.

Encapsulation in Java is a mechanism of wrapping the data (variables) and code acting on the data (methods) together as a single unit. In encapsulation, the variables of a class will be hidden from other classes and can be accessed only through the methods of their current class.

Data Field Encapsulation

To achieve encapsulation in Java:

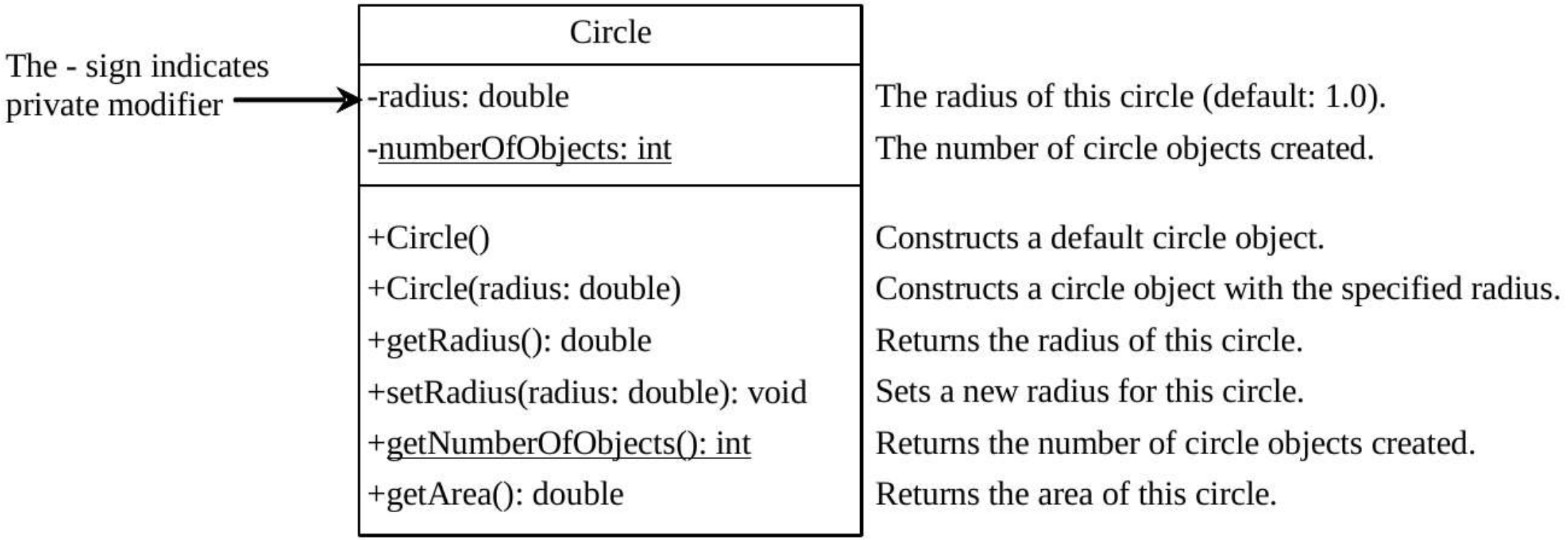
- Declare the variables of a class as private.
  - A private data field cannot be accessed by an object from outside the class
- Provide public setter and getter methods to modify and view the variables values
  - To make a private data field accessible
- A getter method has the following signature:
  - **`public returnType getPropertyName()`**

```
public double getRadius();
```

- A setter method has the following signature:
  - **`public void setPropertyName(dataType propertyValue)`**

```
public void setRadius(double newRadius);
```

Example of Data Field Encapsulation



Array of Objects

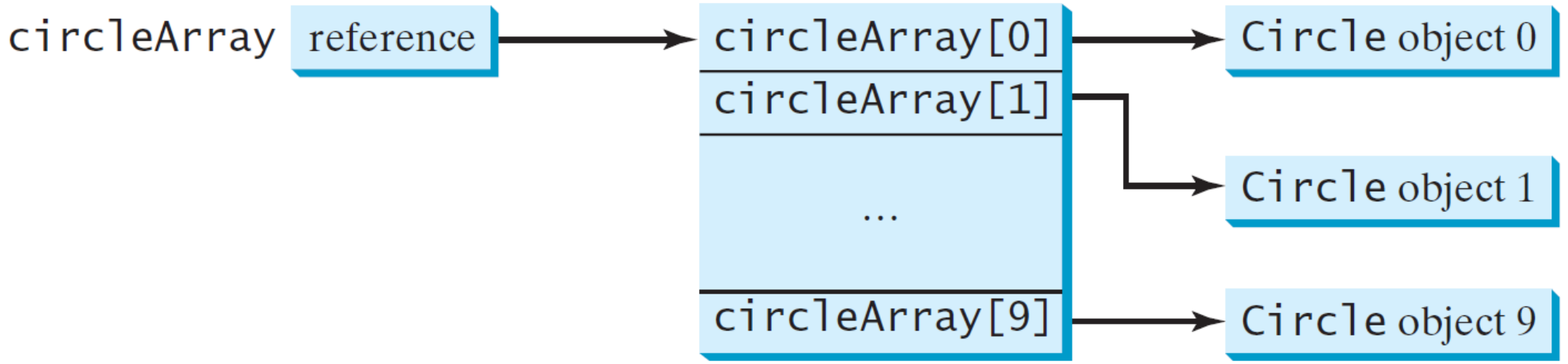
Syntax:

```
Class obj[]= new Class[array_length]
```

Ex:

```
Circle[] circleArray = new Circle[10];
```

An array of objects is actually an array of reference variables.



Arrays are Objects

- An array can contain both primitive types or object references (but no object values)
- In Java an array is an Object
- Array declaration allocates memory space for a reference, whose default value is **`null`**
- Elements are selected with brackets `[ ]` (C-like)
- An array reference is not a pointer to the first element of the array, it is a pointer to the array object

Arrays of Arrays

Since rows are not stored in adjacent positions in memory they can be easily exchanged

```
double[][] balance = new double[5][6];
...
double[] temp = balance[i];
balance[i] = balance[j];
balance[j] = temp;
```

Rows can be created one at a time:

```
int[][] triangle = new int[3][]

for (int i=0; i< triangle.length; i++)
    triangle[i] = new int[i+1];
```

The `this` Keyword

- The **`this`** keyword is the name of a reference that refers to an object itself. One common use of the **`this`** keyword is reference a class's hidden data fields.
- **`this`** makes no sense within methods that have not been invoked on an object, e.g. the **`main`** method

```
public class F {
  private int i = 5;
  private static double k = 0;

  void setI(int i) {
    this.i = i;
  }

  static void setK(double k) {
    F.k = k;
  }
}
```

```
Suppose that f1 and f2 are two objects of F.
F f1 = new F(); F f2 = new F();

Invoking f1.setI(10) is to execute
   this.i = 10, where this refers f1

Invoking f2.setI(45) is to execute
   this.i = 45, where this refers f2
```

Immutable Objects and Classes

If the contents of an object cannot be changed once the object is created, the object is called an immutable object and its class is called an immutable class.

A class with all private data fields and without setters is not necessarily immutable. For example, the following class **`Student`** is mutable.

Example

```
public class Student {
  private int id;
  private BirthDate birthDate;

  public Student(int ssn,
      int year, int month, int day) {
    id = ssn;
    birthDate = new BirthDate(year, month, day);
  }

  public int getId() {
    return id;
  }

  public BirthDate getBirthDate() {
    return birthDate;
  }
}
```

```
public class BirthDate {
  private int year;
  private int month;
  private int day;

  public BirthDate(int newYear,
      int newMonth, int newDay) {
    year = newYear;
    month = newMonth;
    day = newDay;
  }

  public void setYear(int newYear) {
    year = newYear;
  }
}
```

Example, cont.

```
public class Test {
  public static void main(String[] args) {
    Student student = new Student(111223333, 1970, 5, 3);
    BirthDate date = student.getBirthDate();
    date.setYear(2010); // Now the student birth year is changed!
  }
}
```

What Class is Immutable?

For a class to be immutable, it must mark all data fields private and provide no setters and no getters that would return a reference to a mutable data field object.